Vulnerability Assessment of a Live Web Application

Introduction

Web applications today form the backbone of digital services, enabling businesses and individuals to interact, transact, and share information seamlessly. However, this convenience also makes them attractive targets for cyberattacks. Recognizing this reality, a structured vulnerability assessment was conducted on the test site **testaspnet.vulnweb.com**.

The evaluation was performed using **OWASP ZAP (v2.17.0 by Checkmarx)**, a widely respected open-source security testing tool. To ensure credibility and repeatability, the assessment methodology was aligned with two authoritative frameworks: the **OWASP Testing Guide v4** and **NIST SP 800-115**. These frameworks provided a disciplined approach, ensuring that the findings were not only accurate but also actionable.

The primary objective of this project was to uncover potential weaknesses, measure their severity using **CVSS v3.1 scoring**, and recommend practical remediation strategies. The ultimate goal was to demonstrate how proactive security testing can strengthen applications against evolving cyber threats.

Executive Summary

The assessment revealed **no critical or high-severity vulnerabilities**, which is encouraging. Instead, the issues identified were primarily **configuration-level weaknesses** that can be remediated quickly with server hardening and minor code adjustments.

A total of **14 alerts** were generated:

- **4 Medium-level vulnerabilities** (e.g., missing Anti-CSRF tokens, absence of Content Security Policy, lack of anti-clickjacking protection, and no HTTPS enforcement).
- **5 Low-level vulnerabilities** (e.g., insecure cookie attributes, server version disclosure, and header leaks).
- **5 Informational findings**, which highlight minor observations but do not pose immediate risks.

This distribution underscores that the application is not fundamentally flawed but requires attention to **security headers, cookie policies, and HTTPS enforcement**.

Detailed Findings

Medium-Severity Issues

1. **Absence of Anti-CSRF Tokens (CVSS 6.5)**
   Forms such as *about.aspx*, *default.aspx*, and *login.aspx* lacked CSRF protection. Without tokens, attackers could trick users into executing unintended actions.
   Recommendation: Implement OWASP CSRFGuard or ASP.NET's RequestVerificationToken.

2. **Content Security Policy (CSP) Header Not Set (CVSS 5.4)**
   The site did not enforce a CSP header, leaving it vulnerable to cross-site scripting (XSS) and injection attacks.
   Recommendation: Configure a strict CSP header to control script execution and resource loading.

3. **Missing Anti-Clickjacking Header (CVSS 6.1)**
   No X-Frame-Options or CSP frame-ancestors directive was present, exposing users to clickjacking attacks.
   Recommendation: Add X-Frame-Options: DENY or CSP frame-ancestors 'none'.

4. **HTTP Only Site (CVSS 5.0)**
   The application was served only over HTTP, leaving traffic vulnerable to interception.
   Recommendation: Enforce HTTPS redirection and obtain a valid SSL/TLS certificate.

Low-Severity Issues

5. **Cookie Without SameSite Attribute (CVSS 3.5)**
   The ASP.NET_SessionId cookie lacked the SameSite attribute, increasing the risk of CSRF.
   Recommendation: Set SameSite=Lax or Strict.

6. **X-Powered-By Header Leak (CVSS 2.9)**
   The server disclosed its framework via the X-Powered-By: ASP.NET header.
   Recommendation: Suppress this header in IIS configuration.

7. **Server Version Leak (CVSS 3.0)**
   The Server: Microsoft-IIS/8.5 header revealed the exact server version.
   Recommendation: Replace with a generic server header or suppress entirely.

8. **X-AspNet-Version Header Leak (CVSS 3.2)**
   The server disclosed its ASP.NET version (2.0.50727).
   Recommendation: Remove this header to reduce reconnaissance opportunities.

9. **X-Content-Type-Options Header Missing (CVSS 2.5)**
   Anti-MIME sniffing protection was absent.
   Recommendation: Set X-Content-Type-Options: nosniff.

Informational Findings

The scan also highlighted minor issues such as verbose error messages and unnecessary HTTP methods enabled. While these do not pose immediate risks, they provide attackers with additional information that could be leveraged in future attacks.

Analysis

The overall picture is clear: the vulnerabilities identified are **configuration-based** rather than deep application logic flaws. This is a positive outcome, as it means the application can be significantly hardened without major architectural changes.

By implementing missing security headers, enforcing HTTPS, and strengthening cookie policies, the application's resilience against common web attacks will improve dramatically.

Skills Demonstrated

This project showcases practical expertise in:

- **Web Application Security Testing**: Using OWASP ZAP to uncover vulnerabilities.
- **Risk Assessment**: Applying CVSS scoring to prioritize remediation.
- **Security Standards**: Following OWASP and NIST methodologies.
- **Professional Reporting**: Translating technical findings into actionable recommendations.

Conclusion

The assessment emphasizes the importance of **proactive security testing**. Even though no critical vulnerabilities were found, the medium and low-level issues represent potential entry points for attackers if left unresolved.

By addressing these configuration weaknesses, organizations can significantly reduce their exposure to threats. This project demonstrates that **security is not a one-time effort but an ongoing process**, requiring vigilance, regular assessments, and adherence to best practices.